

PQ-STAR: Post-Quantum Stateless Auditable Rekeying [★]

(Preliminary Version)

Shlomi Dolev¹, Avraham Yagudaev¹, and Moti Yung²

¹ Department of Computer Science
Ben-Gurion University of the Negev
Beer Sheva, Israel

dolev@cs.bgu.ac.il, avraamy@post.bgu.ac.il

² Google, USA
motiyung@gmail.com

Abstract. Rekeying is an effective technique for protecting symmetric ciphers against side-channel and key-search attacks. Since its introduction, numerous rekeying schemes have been developed [39]. We introduce *Post-Quantum Stateless Auditable Rekeying (PQ-STAR)*, a novel post-quantum secure [12] stateless rekeying scheme with *audit* support. PQ-STAR is presented in three variants of increasing security guarantees: (i) *Plain PQ-STAR* lets an authorized auditor decrypt and verify selected ciphertexts; (ii) *Commitment-based PQ-STAR* with the additional binding guarantee from the commitments, preventing a malicious sender from potentially claiming a random or wrong session key. (iii) *Zero-knowledge PQ-STAR* equips each session key with a signature-based zero-knowledge proof (ZKP), which proves that the session key was derived honestly, without ever revealing the secret preimage. We formally prove that all variants achieve *key-uniqueness*, *index-hiding*, and *forward-secrecy*, even if a probabilistic polynomial-time (PPT) adversary arbitrarily learns many past session keys. PQ-STAR provides a formally verified, stateless, and audit-capable rekeying primitive that can be seamlessly integrated as a post-quantum upgrade for existing symmetric-key infrastructures.

Keywords: Post-Quantum Cryptography · Stateless Rekeying · Side-Channel Resistance · Symmetric-Key Updates · Zero-Knowledge Proof · Auditability

1 Introduction

Side-channel attacks [32] endanger practical cryptographic implementations by exploiting physical leakages, such as timing, power consumption, or electromagnetic emissions. A well-established countermeasure is rekeying, i.e., updating

[★] Partially supported by the Google Research Grant, the Rita Altura Trust Chair in Computer Science, the BGU Data Center, the Frankel Center for Computer Science, and the Israeli Science Foundation (Grant No. 465/22).

cryptographic keys at sufficiently short intervals so that an adversary observes only negligible leakage for any single key.

Rekeying [2] can be categorized as either *stateless (parallel)* or *stateful (sequential)*. Stateless rekeying - also called *parallel rekeying* [1,2] - derives every session key directly from a long-term master key, which must withstand both Simple Power Analysis (SPA) and Differential Power Analysis (DPA) [24,25]. In contrast, stateful rekeying maintains a mutable state for generating subsequent keys and was popularized by Kocher’s index-based scheme [26].

Recently, Chang *et al.* [10] introduced HOP-1 and HOP-2 as refinements of Kocher’s design. Whereas the original permits up to three encryptions per session key, HOP-1 restricts a key to two uses, and HOP-2 forbids any reuse.

In a related direction, Dolev *et al.* proposed HBSS [17], a post-quantum, hash-based stateless signature scheme. HBSS combines Lamport signatures, Merkle trees, and Bloom filters to authenticate many messages without maintaining state. Although HBSS addresses digital signatures rather than key evolution, its stateless, hash-based nature motivated the design choices of PQ-STAR.

1.1 Our Contributions

We introduce PQ-STAR, the first rekeying primitive that unifies post-quantum security, stateless fresh rekeying, and an explicit audit mechanism. Auditability is beneficial in various social contexts, as it can provide (sampled) evidence of correct and lawful actions, see, e.g., [16]. We present three progressively stronger variants: plain, commitment-based, and zero-knowledge proof, and prove that each satisfies *key-uniqueness*, *index-hiding*, and *forward-secrecy*. We also provide parameter choices and resource costs for PQ-STAR.

1.2 Notation

- We denote m as the size of the shared secret array, and $A = (a_1, \dots, a_m)$ as the master secret array (shared between Alice and Bob). Each entry $a_i \in \{0, 1\}^n$ is an n -bit string.
- k is the number of indices selected in each session, $\mathcal{I} = (j_1, \dots, j_k)$ is an index tuple with each $j_i \in [m] := \{1, 2, \dots, m\}$, and $B = (a_{j_1}, \dots, a_{j_k})$ is the tuple of selected secret values.
- We compute the session key as $sk = H(a_{j_1} \parallel \dots \parallel a_{j_k})$, where $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is a cryptographic hash function (e.g., SHA-256) and $\parallel$ denotes bit-string concatenation.
- We denote M as a plaintext message, C as ciphertext, and r as a random nonce. We denote the functions $\text{ENC}_{sk}(\cdot; r)$ (and $\text{DEC}_{sk}(\cdot)$) as encryption (and decryption) with key sk .
- $Cmt = (c_1, \dots, c_m)$ is the public commitment array, where $c_i = \text{SHA}(a_i)$.
- σ denotes a FAEST signature, and $(\text{pk}_{\text{FAEST}}, \text{sk}_{\text{FAEST}})$ is the FAEST key pair.

1.3 Paper Organization

The paper is organized as follows. Section 2 reviews related work. Section 3 details the plain PQ-STAR scheme, Section 4 and Section 5 present the commitment-based and zero-knowledge variants, respectively. Section 6 discusses performance and parameter selection. Finally, Section 7 concludes.

2 Related Work

2.1 Rekeying and Side-Channel Security

Rekeying was first studied to extend the secure lifetime of symmetric keys. Abdalla and Bellare [2] formalized this idea, showing that deriving short-lived session keys from a long-term master key can *increase the lifetime of a key* and mitigate classical cryptanalysis limits such as differential or linear attacks [30] and birthday collisions. Independently, Kocher’s introduction of Differential Power Analysis (DPA) in 1999 [24] noted that simply updating the secret key frequently would render many power-analysis attacks ineffective. As [2] observes, Kocher et al. proposed changing the key *for each or after a small number of encryptions* - a method known as *rekeying*. In practice, this means each encryption uses a fresh key, preventing attackers from collecting large trace sets on any fixed key.

Early applications of this concept were seen in cryptographic protocols and standards that aimed to limit the exposure of long-term keys, and it was soon adopted in hardware security as a lightweight countermeasure against DPA. Seminal work established that rekeying can provide provable security gains without changing the underlying cipher [2]. Later research confirmed its efficacy against side-channel adversaries when combined with a secure key-update function [1].

The principal benefit of rekeying is that it caps the statistical advantage an attacker gains by correlating many leakages on the same key. By refreshing keys faster than an attacker can exploit leaked information, rekeying dramatically raises the effort required for DPA-style attacks: an adversary can no longer average hundreds or thousands of traces on the same key, since the key changes frequently. Rekeying complements masking, which injects randomness into intermediate values to decorrelate power consumption from secrets [13]; leakage-resilient designs that bound the total information extracted by an adversary [19]; and hardware-level hiding techniques such as balanced dual-rail logic and noise injection, which reduce observable side-channel signals at the circuit level [40]. Masking and hiding countermeasures require specialized hardware or algorithmic changes, while rekeying adds only a key-update step and leaves the underlying cipher untouched, making it attractive for retrofitting into existing systems.

2.2 Rekeying Variants and Recent Work

In its simplest form, fresh rekeying generates each new session key from the previous one (or from a counter) under a master key. [1] describes two modes: applying a pseudorandom function to an index in parallel, or deriving each key

sequentially from the previous one. The latter is best suited to avoid DPA, as it inherently changes the key on every encryption; however, the method faces an efficiency problem when a client and a server need to resynchronize.

Research on rekeying has progressed steadily: early key-derivation proposals of the 2000s evolved into recent leakage-resistant designs that rely on frequent key rotation to blunt side-channel attacks [1,33]. Because rekeying periodically refreshes the secret state, it offers a practical compromise between security and performance: the key is refreshed often enough to narrow an attacker’s window, yet the added cost stays modest.

The idea of fresh (frequent) rekeying has continued to attract attention. Medwed *et al.* [33] formalized fresh rekeying for lightweight protocols in 2010, and later papers introduced more efficient instantiations together with complete security proofs. For instance, Mennink [34] presents generalized rekeying constructions that, for the first time, achieve beyond-birthday-bound security against side channels. [41] proposes an LR4 rekeying mode with a formal leakage-resistant proof.

See also [39] for a specification of symmetric-key rekeying mechanisms.

2.3 Indexed Rekeying Schemes (Kocher, HOP-1, HOP-2)

A concrete approach to rekeying is *indexed sequential rekeying*, developed by Kocher [26]. Kocher’s scheme requires the sender to maintain a secret current key, updating it after every message, while the receiver remains stateless by deriving any session key directly from the message index. The procedure resembles a traversal of a binary key tree rooted at the long-term secret: each message either uses the current node’s key or moves to a child node through two update functions, usually denoted F_A and F_B . Kocher specified F_A, F_B , and their inverses so that each key node appears in at most three sessions.

Subsequent work [10] refines the traversal. *HOP-1* prevents session-key reuse by introducing a composite update; *HOP-2* strengthens the guarantee to single-use keys. Satisfying the HOP-2 property requires seven transitions, compared with four in Kocher’s scheme, and effectively performs an Eulerian walk of the key tree. Consequently, every key under HOP-2 contributes to exactly one ciphertext, eliminating reuse.

Kocher’s design remains influential because it trades modest per-message computation for a dramatic reduction in key reuse. Although the scheme lacks a formal proof, fragmenting key usage frustrates classical DPA. As with other fresh-rekeying modes, indexed rekeying does not eliminate single-trace (SPA) leakage; however, it severely limits the data available to any multi-trace attack.

2.4 Post-Quantum Signatures and Stateless Schemes

We have concentrated on symmetric and key-exchange primitives. Concurrently, the cryptography community has turned its attention to quantum adversaries. According to NIST’s assessment [12], large-scale quantum computers would

endanger many existing public-key systems, motivating the search for post-quantum alternatives. In the post-quantum setting, symmetric primitives and hash functions remain secure - Grover’s algorithm [20] offers only a quadratic speed-up, which doubling key lengths can counter - so schemes that depend solely on hashes look promising.

A prominent family of post-quantum signatures is the hash-based family. Hash-based signatures build on one-time signing techniques such as Lamport’s scheme [29] and typically employ a Merkle tree to enable many signatures. Because the security argument requires only collision-resistant hashes, no number-theoretic assumptions arise. The primary drawback is statefulness: the signer must track whether signatures have been issued.

Recent work removes this restriction by designing stateless hash-based signatures. Dolev *et al.* [17] introduce HBSS, a stateless hash-based signature scheme. HBSS asks the signer to precompute an array of m secret values (preimages) and to publish their hashes as a commitment array. To sign a message, the signer discloses a subset of these preimages determined by the message’s hash digest. For instance, with a 512-bit hash and an array of size m , each signature reveals exactly 512 array entries. Selecting m sufficiently large allows HBSS to support a predetermined number of signatures without collision.

NIST has standardized SPHINCS+ [6], a stateless hash-based signature scheme designed for post-quantum security. While SPHINCS+ offers a flexible and well-analyzed framework, our use of HBSS emphasizes simplicity and auditability. HBSS uses a Merkle tree and Bloom filters to enable many one-time signatures without maintaining state, and its commitment array naturally aligns with PQ-STAR’s index selection mechanism. In scenarios where deployment simplicity and verifiable randomness are paramount (such as our audit use case), HBSS integrates cleanly with PQ-STAR. SPHINCS+, on the other hand, targets general-purpose digital signatures and a more involved structure; it remains a strong candidate for standalone signatures but is not directly tailored to the audit functionality considered here.

3 Plain PQ-STAR

3.1 Construction

PQ-STAR is a stateless rekeying scheme that supports message auditing by a third party. The scheme relies on a large shared secret array from which ephemeral session keys are derived. By design, each encrypted message can be independently verified by an auditor without revealing any long-term secret.

Let $m \geq 1$ and let $A = (a_1, a_2, \dots, a_m)$ be an array of m secret values shared between Alice and Bob (e.g., via an initial key exchange). The array A functions as a large shared secret key that must remain confidential at all times (like any long-term symmetric key). Because A may be very large, it is practical to generate each a_i pseudorandomly from a short seed (or seeds) instead of storing the entire array. For example, given two seeds $seed_l$ and $seed_r$, one may set

$$a_i := \text{SHA}^i(\text{seed}_r) \oplus \text{SHA}^{(m-i+1)}(\text{seed}_l),$$

so that each a_i is derived from both seeds (the left seed seed_l is iterated forward and the right seed seed_r is iterated backward), this construction behaves like a “lock”: with only a single seed, once one preimage is exposed, subsequent preimage values can be computed by iterative hashing. This obviates the necessity to store the full array A ; any entry can be recomputed on demand from the seeds. This approach is similar to standard techniques for expanding a one-time pad from a short seed in a post-quantum setting [4,15,17]. A time-memory trade-off (e.g., precomputing $O(\sqrt{m})$ intermediate values) can further reduce the computation cost. This construction ensures that reconstructing any a_i requires both seeds. Note that a similar locking can be based on a single seed, as follows, $a_i := \text{SHA}^i(\text{seed}) \oplus \text{SHA}^{(i+1)}(\text{seed})$. Another approach for deriving each a_i from a single seed and its index i is based on *key-derivation function* (KDF) [11,23,27,28]. One may simply use $a_i = \text{AES}(\text{seed} \parallel i)$ in contexts where the risk of breaking the cipher is considered small [3,7,8,36].

In PQ-STAR (Figure 1), each session (each message sent between Alice and Bob) uses a fresh session key derived from A ; the complete procedure appears in Algorithm 1. At a high level, the process is as follows:

1. Alice and Bob establish the shared master array A (or equivalent seeds).
2. For every new message, Alice (or Bob) selects k random indices (with repetition allowed) to assemble an index tuple $\mathcal{I} = (j_1, \dots, j_k)$ from $[m]$. She then computes the session key

$$sk = H(a_{j_1} \parallel a_{j_2} \parallel \dots \parallel a_{j_k}),$$

where H is a cryptographic hash function (e.g., SHA-256). Bob (or Alice), who also knows A , can derive the identical sk from the shared tuple $\mathcal{I}$. We intentionally use a one-way hash to derive sk rather than a reversible block cipher: symmetric block ciphers instantiate pseudorandom permutations, which are invertible given the key, so an adversary with the encryption key could invert the mapping and recover the secret inputs used to derive sk . A cryptographic hash compresses its input and resists preimage and collision attacks, guaranteeing that the selected array entries remain confidential.

3. Alice uses sk to encrypt the message, and she includes $\mathcal{I}$ in the transmission (in plaintext) so that Bob knows which a_i ’s were used. An authorized auditor can later verify the ciphertext using $\mathcal{I}$ and sk (as described below).

Expanding steps 2 and 3, let $\mathcal{I} = (j_1, \dots, j_k)$ be the indices chosen for a message M , and let $B = (a_{j_1}, \dots, a_{j_k})$ be the corresponding secret values. The index tuple $\mathcal{I}$ is made public with the ciphertext, while B remains known only to Alice and Bob. Alice computes the session key $sk = H(B)$ and then encrypts M using sk . To prevent an eavesdropper from recognizing repeat messages, the encryption must be randomized with a fresh random *nonce* for each encryption.

We denote the encryption of M (with associated data $\mathcal{I}$) under key sk and nonce r by

$$C = \text{ENC}_{sk}(M \parallel \mathcal{I}; r).$$

Alice sends the pair $(C, \mathcal{I})$ to Bob. Upon receiving it, Bob recomputes $sk = H(a_{j_1}, \dots, a_{j_k})$ using his copy of A , and decrypts C with sk to recover M . Because of the random nonce, encrypting identical messages (even with the same $\mathcal{I}$) yields different ciphertexts, preventing pattern recognition by an eavesdropper.

Remark. An alternative scheme (proposed by an anonymous reviewer) is based on AES as follows. To encrypt a message M , Alice randomly selects a nonce r and uses AES with the master key A to encrypt r , obtaining a session key sk for M . She then sends r concatenated with $\text{AES}_{sk}(r \parallel M)$, namely,

$$r \parallel \text{AES}_{sk}(r \parallel M).$$

In the sequel, during the audit process, we require authentication of Alice and Bob, and the resulting schemes are more involved.

PQ-STAR also supports an *audit* mechanism to prove the origin and correctness of an encrypted message. If an authorized auditor needs to verify a ciphertext C and its claimed plaintext M , Alice (or Bob) can reveal the corresponding session key sk without exposing any part of the master key A . Concretely, Alice provides the auditor with the tuple $(C, \mathcal{I})$ and the session key sk used for C . The auditor decrypts C with sk , obtaining $M' \parallel \mathcal{I}' = \text{DEC}_{sk}(C)$. If $\mathcal{I}' = \mathcal{I}$ (we conclude that then M' matches the claimed M), then C is confirmed as the encryption of M under the key derived from $\mathcal{I}$. This audit procedure confirms the authenticity of M without revealing any long-term secret in A (only the one-time key sk). Even a malicious auditor learns nothing beyond the disclosed sk , which (as shown in the security analysis) does not compromise the scheme.

In practice, the *MasterKey* setup is executed once at initialization. Thereafter, Alice and Bob derive new session keys on the fly as needed, without any further key exchanges. Each invocation of *EncryptMessage* uses a fresh random index tuple and nonce, ensuring that session keys and ciphertexts are independent across sessions. Importantly, neither party needs to store the full array A in memory permanently; given the shared seed(s), any required entry a_i can be recomputed on demand from the seeds.

3.2 Security Analysis

We first build intuition with three boundary cases:

Example 1 ($n = 1$). Each a_i is one bit long. For instance, if $m = 2$ and $k = 1$, the probability that two different sessions use the same key is half. This trivial case illustrates the need for a sufficiently large n to avoid frequent key collisions.

Example 2 ($k = 1$). Each session key is $sk = H(a_{j_1})$, derived from a single array entry. In this case, there are at most m possible session keys, and by the birthday

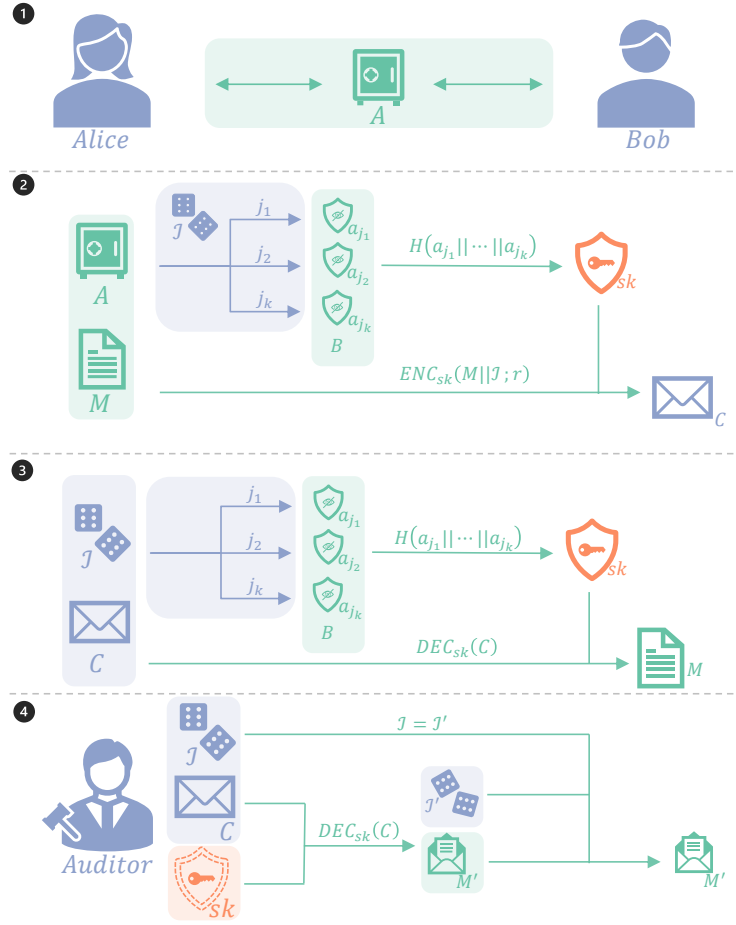


Fig. 1. Overview flow of the Plain PQ-STAR scheme.

paradox attack, the probability of reusing the same key is $\sqrt{1/m}$, which may not be negligible. This highlights the advantage of choosing $k > 1$, which provides exponentially many possible ordered key combinations.

The next example motivates the use of a one-way function applied to preimages of the chosen entries instead of XORing them (which is order agnostic).

Example 3 ($m = 4$). Suppose each a_i is an n -bit string. One might try deriving session keys by XORing two entries (e.g., $a_i \oplus a_j$), but this yields only seven distinct outcomes, whereas two independent n -bit values can produce 2^n outcomes (e.g., 16 if $n = 4$). The XOR method fails to achieve a large or uniform key space. It also introduces a special case $a_i \oplus a_i = 0$ when the same index is used twice, a pattern that could be exploited by side-channel analysis. A better

Algorithm 1 PQ-STAR auditable rekeying scheme

```

1: function MasterKey():
2:    $A \leftarrow \text{array}[m]$  //  $A = (a_1, \dots, a_m)$ 
3:   for  $i \leftarrow 1$  to  $m$  do
4:      $a_i \leftarrow \text{Random}(0, 2^n - 1)$  // choose a random  $n$ -bit value
5:   end for
6:   return  $A$ 
7: end function
8: function SessionKey( $A, \mathcal{I}$ ):
9:    $B \leftarrow \text{array}[k]$  //  $B = (b_1, \dots, b_k)$ 
10:  for  $i \leftarrow 1$  to  $k$  do
11:     $b_i \leftarrow A[\mathcal{I}[i]]$ 
12:  end for
13:   $sk \leftarrow H(b_1 \parallel \dots \parallel b_k)$  // derive session key
14:  return  $sk$ 
15: end function
16: function EncryptMessage( $A, M$ ):
17:    $\mathcal{I} \leftarrow \text{array}[k]$  //  $\mathcal{I} = (j_1, \dots, j_k)$ 
18:   for  $i \leftarrow 1$  to  $k$  do
19:      $j_i \leftarrow \text{Random}(1, m)$  // uniform random index
20:   end for
21:    $sk \leftarrow \text{SessionKey}(A, \mathcal{I})$ 
22:    $r \leftarrow \text{Random}(0, 2^n - 1)$  // random  $n$ -bit nonce
23:    $C \leftarrow \text{ENC}_{sk}(M \parallel \mathcal{I}; r)$ 
24:   return  $(C, \mathcal{I})$ 
25: end function
26: function Audit( $C, \mathcal{I}, sk$ ):
27:    $M' \parallel \mathcal{I}' \leftarrow \text{DEC}_{sk}(C)$ 
28:   return  $M'$  and  $\mathcal{I}' = \mathcal{I}$ 

```

approach is to use a one-way function on the tuple of preimage values (as we do) to avoid these issues.

Recall that $A = (a_1, \dots, a_m)$ be the shared master array with each $a_i \in \{0, 1\}^n$ chosen independently and uniformly at random. Each session chooses a fresh random index tuple $\mathcal{I} = (j_1, \dots, j_k)$ and sets $B = (a_{j_1}, \dots, a_{j_k})$. Since the a_i are uniform, for any fixed $\mathcal{I}$ the concatenated string B is uniformly distributed in $\{0, 1\}^{kn}$. We assume the hash $H : \{0, 1\}^{kn} \rightarrow \{0, 1\}^n$ is a standard post-quantum cryptographic hash (e.g., SHA-256) with one-way and pseudorandomness properties. Then the session key $sk = H(B)$ is effectively a random n -bit string from the PPT adversary's viewpoint. We now establish three security properties.

The next proofs holds for large enough parameters; for instance $n \geq 256$, $k \geq 16$, and $m \geq 2^{20}$.

Theorem 1 (Key-Uniqueness). *With overwhelming probability, distinct sessions produce distinct session keys.*

Proof. Two distinct sessions can produce the same session key sk only if either they happen to choose to use the identical index tuple $\mathcal{I}$ or if the hash function H outputs the same value on two different inputs (i.e., collision). Since each session chooses $\mathcal{I}$ independently, the probability that two sessions pick the same tuple is m^{-k} , which is negligible when m^k is superpolynomial. Over any polynomial number of sessions, a union bound implies that the probability of any index-tuple collision remains negligible. If two sessions use different tuples, then with overwhelming probability $B \neq B'$. In that case, obtaining sk from B and B' would require a hash collision ($H(B) = H(B')$). By the collision-resistance of H , the probability of such a collision is also negligible. Hence, except with negligible probability, distinct sessions yield distinct keys. $\square$

Theorem 2 (Index-Hiding). *No PPT adversary has more than a negligible advantage in determining the session key sk from the public index tuple $\mathcal{I}$.*

Proof. The adversary observes $\mathcal{I} = (j_1, \dots, j_k)$ but not the actual secret values $a_{j_1}, \dots, a_{j_k}$. To the adversary, the tuple $B = (a_{j_1}, \dots, a_{j_k})$ is a uniformly random string in $\{0, 1\}^{kn}$. Thus, the input to H is unpredictable. Consequently, $H(B)$ produces a pseudorandom n -bit output under our hash assumptions. In particular, knowing $\mathcal{I}$ gives no advantage in guessing $sk = H(B)$: since the hash output is indistinguishable from a uniform n -bit string. Even if the adversary tries to guess one of the secret values (say a_{j_ℓ} for some $\ell \in \{1, \dots, k\}$), any such guess has at most a 2^{-n} chance of yielding the correct sk . By the pseudorandomness and preimage resistance of H , no polynomial-time adversary can do significantly better than random guessing. Therefore, the observed $\mathcal{I}$ and the associated ciphertext leak no information about sk . $\square$

Theorem 3 (Forward-Security). *No PPT adversary that obtains up to t session keys (and their index tuples) has more than negligible advantage in learning any information about any other session key.*

Proof. Suppose the adversary learns $(\mathcal{I}_1, sk_1), \dots, (\mathcal{I}_t, sk_t)$ for some sessions. This information alone reveals nothing about the underlying master secrets. In particular, each $sk_j = H(a_{j_1} \parallel \dots \parallel a_{j_k})$ is the output of a one-way hash function, so the adversary learns no direct information about any individual a_i . Any index i that does not appear in the union $\bigcup_{j=1}^t \mathcal{I}_j$ remains completely unknown to the adversary. Even if some index i appears in the revealed sessions, the adversary learns only a hash value involving a_i , not the secret a_i itself.

Now consider a fresh session with index tuple $\mathcal{I}'$ and input B' . Some indices in $\mathcal{I}'$ may overlap with previous tuples, but the adversary still does not know the actual secret values for those indices. Moreover, since we assume $t = \text{poly}(m)$ (as any PPT adversary can obtain only polynomially many sessions), and the possible number of session keys is exponential, namely, m^k , there are many session keys that remain unused and random. Because the adversary does not know the input B' , by the pseudorandomness of H , the new key $sk' = H(B')$ is independent of all previously revealed keys. Formally, any polynomial-time adversary that sees $\{(\mathcal{I}_j, sk_j)\}_{j=1}^t$ has only negligible advantage in predicting or distinguishing sk' for an unseen $\mathcal{I}'$ (otherwise it would break the pseudorandomness

of H). Thus, revealing a bounded number of session keys does not compromise the security of other session keys. $\square$

Taken together, three Theorems 1 to 3 imply that PQ-STAR’s session keys are (i) unique across sessions except with negligible probability, (ii) independent of the public index information, and (iii) secure even if some session keys are revealed.

This establishes the intended post-quantum security properties of the scheme.

On key commitment. Our security proofs implicitly assume that the authenticated encryption primitive binds each ciphertext to a unique key. This assumption does not hold for many widely deployed *authenticated encryption with associated data* (AEAD) modes, for example, AES-GCM [18] and ChaCha20-Poly1305 [35] can decrypt a single ciphertext under multiple keys, enabling *partitioning-oracle attacks* in which a malicious sender crafts a ciphertext that is valid under two different keys [22, 31]. Such attacks would undermine PQ-STAR’s audit functionality, because an auditor checking $\text{DEC}_{sk}(C)$ could not distinguish the honest key sk from an adversarial key. To prevent this, the encryption algorithm used with Plain PQ-STAR must be *key-committing*: for any fixed ciphertext and associated data, at most one key yields a valid decryption. Key-committing AEAD (or a generic Encrypt-then-MAC with a single key) mitigates this risk [9]. Some modern designs (e.g., AEGIS) provide forms of committing security under specific conditions; care is required when adversaries control associated data [14]. These designs achieve this property by design and therefore resist partitioning-oracle attacks. Alternatively, one can enforce key commitment by including a commitment tag in the plaintext or by computing a separate hash-based MAC over the ciphertext and associated data. In particular, the commitment-based and zero-knowledge variants inherently provide such binding information (public commitments or a signature on the session key). They therefore guarantee the key-commitment property and are robust against partitioning-oracle attacks.

4 Commitment-Based PQ-STAR

4.1 Construction

The commitment-based variant of PQ-STAR differs from the plain scheme only in the *Setup* and *Audit* phases (Figure 2). As before, Alice and Bob share a secret array $A = (a_1, \dots, a_m)$ of random n -bit values. In *Setup*, they derive a *commitment* array $Cmt = (c_1, \dots, c_m)$ by setting

$$c_i = \text{SHA}(a_i),$$

(or, equivalently, derive a single Merkle tree root R as in HBSS [17]; any post-quantum collision-resistant commitment works).

where SHA is a cryptographic hash function (e.g., SHA-256). The Cmt (or root R) is published publicly, while the values a_i remain secret. These commitments bind Alice and Bob to their chosen preimages (the a_i): once c_i is public, neither party can later claim a different a_i without detection (this is the *binding* property of a hash-based commitment).

The *Session Key Derivation* and *Encryption* steps are the same as in plain PQ-STAR. For each message, Alice chooses (with possible repetition) k random indices $\mathcal{I} = (j_1, \dots, j_k) \in [m]^k$. She retrieves the corresponding secret values $a_{j_1}, \dots, a_{j_k}$, computes the session key

$$sk = H(a_{j_1} \parallel a_{j_2} \parallel \dots \parallel a_{j_k}),$$

and uses sk to encrypt her message M , including $\mathcal{I}$ as associated data (with a fresh random nonce). She sends the ciphertext C together with $\mathcal{I}$ to Bob. Bob, knowing A , recomputes sk from $\mathcal{I}$ and decrypts C to recover M . (This part is unchanged from Section 3.)

The *Audit* procedure now makes use of the commitments C . If a trusted auditor needs to verify that ciphertext C encrypts message M under the key derived from $\mathcal{I}$, Alice (or Bob) reveals the claimed session key sk and the corresponding preimage values $a_{j_1}, \dots, a_{j_k}$ to the auditor. The auditor then performs two checks:

1. For each $\ell = 1, \dots, k$, the auditor verifies that $\text{SHA}(a_{j_\ell}) = c_{j_\ell}$, using the public commitment c_{j_ℓ} . This ensures the revealed a_{j_ℓ} truly matches the earlier commitment (the communicating parties were actually Alice and Bob (and not, for example, Carol and David)).
2. The auditor verifies that $H(a_{j_1} \parallel \dots \parallel a_{j_k}) = sk$, confirming that the revealed values indeed generate the announced session key.

Finally, the auditor decrypts C under sk and checks that the decrypted plaintext is M with index tuple $\mathcal{I}$. If all checks pass, the audit succeeds. Otherwise, the auditor rejects the transcript. In summary, the auditor's verification now includes checking each revealed preimage against its hash commitment and re-deriving sk , much like the verification in an HBSS signature scheme [17]. Because the hash commitments are binding, Alice (or Bob) cannot cheat in the audit by supplying inconsistent values without being caught.

In Algorithm 2, the MasterKey setup is extended to compute commitments, and an *Audit* function is extended to verify commitments. These additions ensure correctness without repeating the entire scheme. In effect, the committed scheme's setup now includes the one-way hashes (commitments) of all preimages, and the audit function implements the two extra consistency checks described above.

4.2 Security Analysis

Note that in Section 3 we assumed no preimage values were ever revealed (since there was no audit). In the commitment variant, each audit reveals the specific

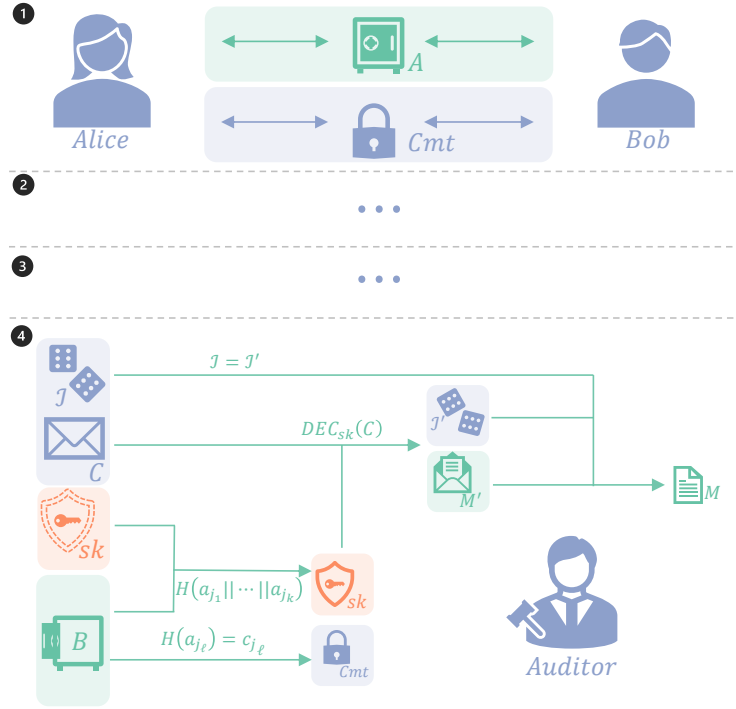


Fig. 2. Overview of the commitment-based PQ-STAR scheme.

a_i values used in that session. Thus, the total number of sessions derived from a single master key must be further limited to preserve the unpredictability of new session keys. In practice, one can adopt the technique from HBSS [17] where the parameter α represents the fraction of total array entries that have been exposed so far. For instance, one might stop generating new session keys once the last audit causes $\alpha \geq 0.5$, ensuring that in every remaining session at least half of the preimages are unseen so far. This maintains the scheme's index-hiding and forward-secrecy (because the adversary must still guess at least unseen $k/2$ new preimages).

Arguments similar to the ones in Theorems 1 to 3, Section 3 implies key-uniqueness, index-hiding, and forward-secrecy. Those proofs remain valid for the commitment variant, with the additional binding guarantee from the commitments.

Theorem 4 (Computational Binding). *No PPT malicious sender can successfully claim a random session key or an incorrect set of preimages without detection, except with negligible probability.*

Proof. In the plain scheme, a malicious sender could potentially claim a random session key or a different set of preimages without being detected immediately

Algorithm 2 Commitment-based PQ-STAR auditable rekeying scheme

```

1: function MasterKey():
2:    $A \leftarrow \text{array}[m]$ 
3:    $Cmt \leftarrow \text{array}[m]$                                      //  $Cmt = (c_1, \dots, c_m)$ 
4:   for  $i \leftarrow 1$  to  $m$  do:
5:      $a_i \leftarrow \text{Random}(0, 2^n - 1)$ 
6:      $c_i \leftarrow \text{SHA}(a_i)$                                      // commitment
7:   end for
8:   return  $(A, Cmt)$                                            //  $Cmt$  is made public
9: end function
10: function Audit( $C, \mathcal{I}, sk, a_{j_1}, \dots, a_{j_k}$ ):
11:   for  $\ell \leftarrow 1$  to  $k$  do
12:     if  $\text{SHA}(a_{j_\ell}) \neq c_{j_\ell}$  then return false
13:   end for
14:   if  $H(a_{j_1} \parallel \dots \parallel a_{j_k}) \neq sk$  then return false
15:    $M' \parallel \mathcal{I}' \leftarrow \text{DEC}_{sk}(C)$ 
16:   return  $M'$  and  $\mathcal{I}' = \mathcal{I}$ 
17: end function

```

by a third-party auditor. With commitments, this becomes infeasible (assuming the hash cryptographic function H is collision-resistant). Any attempt to use a false preimage $a'_{j_\ell} \neq a_{j_\ell}$ will (except with negligible probability) be caught by the auditor's check $H(a'_{j_\ell}) = c_{j_\ell}$, because finding a second preimage that maps to c_{j_ℓ} is infeasible. Thus, a cheating party cannot substitute an incorrect key or preimage set without the auditor noticing.

Formally, the commitments $c_i = \text{SHA}(a_i)$ bind Alice and Bob to their master array: once c_i is public, revealing any a_i inconsistent with c_i would break the binding property of the hash-based commitment. As a result, if an audit succeeds, the auditor is assured that the disclosed sk and preimages are guaranteed (under the hash's one-wayness of H) to be derived honestly from the published indices. $\square$

In summary, adding commitments preserves all the post-quantum security properties of plain PQ-STAR while preventing the potential consistency attacks discussed above.

5 Zero-Knowledge-Based PQ-STAR

5.1 Construction

We now describe a third variant of PQ-STAR that enables an auditor to *validate* every session key in *zero knowledge*. Instead of exposing the preimages $a_{j_1}, \dots, a_{j_k}$ during the audit (as in Section 4), the sender proves possession of the correct secret material using a post-quantum digital signature equipped with a zero-knowledge proof. We instantiate the signature with *FAEST* [5], but any (post-quantum) scheme providing unforgeability and zero-knowledge suffices.

Recall $A = (a_1, \dots, a_m)$ denotes the shared master array, let $(\text{pk}_{\text{FAEST}}, \text{sk}_{\text{FAEST}})$ be a FAEST key pair generated so that the public key implicitly commits to A . The message flow mirrors the Section 3 except that the ciphertext additionally authenticates the derived session key.

Given an index tuple $\mathcal{I} = (j_1, \dots, j_k)$, Alice and Bob compute

$$sk = H(a_{j_1} \parallel \dots \parallel a_{j_k}).$$

Alice (or Bob) then produces a FAEST signature $\sigma \leftarrow \text{Sign}_{\text{FAEST}}(\text{sk}_{\text{FAEST}}, sk)$ and transmits

$$C = \text{ENC}_{sk}(M \parallel \mathcal{I} \parallel \sigma; r),$$

where r is a fresh random nonce, Bob (or Alice) decrypts as usual using the locally derived key sk .

To verify a transcript $(C, \mathcal{I})$, an auditor proceeds as follows:

1. Alice reveals the session key sk .
2. The auditor decrypts C with sk to obtain $(M', \mathcal{I}', \sigma')$ and aborts if $\mathcal{I}' \neq \mathcal{I}$.
3. The auditor checks that $\text{Verify}_{\text{FAEST}}(\text{pk}_{\text{FAEST}}, sk, \sigma') = 1$.

At no point does the auditor learn any a_i ; the zero-knowledge property of FAEST guarantees that the signature discloses no information beyond the validity of sk . Algorithm 3 highlights the additional steps relative to Algorithm 1. In particular, the commitment is by using FAEST public key, as described in line 7.

Algorithm 3 Zero-knowledge PQ-STAR

```

1: function EncryptMessage( $A, M$ ):
2:    $\mathcal{I} \leftarrow \text{array}[k]$ 
3:   for  $i \leftarrow 1$  to  $k$  do
4:      $j_i \leftarrow \text{Random}(1, m)$ 
5:   end for
6:    $sk \leftarrow \text{SESSIONKEY}(A, \mathcal{I})$ 
7:    $\sigma \leftarrow \text{Sign}_{\text{FAEST}}(\text{sk}_{\text{FAEST}}, sk)$ 
8:    $r \leftarrow \text{Random}(0, 2^n - 1)$ 
9:    $C \leftarrow \text{ENC}_{sk}(M \parallel \mathcal{I} \parallel \sigma; r)$ 
10:  return  $(C, \mathcal{I})$ 
11: end function
12: function Audit( $C, \mathcal{I}, sk$ ):
13:    $M' \parallel \mathcal{I}' \parallel \sigma' \leftarrow \text{DEC}_{sk}(C)$ 
14:   if  $\mathcal{I}' \neq \mathcal{I}$  then return false
15:   if  $\text{Verify}_{\text{FAEST}}(\text{pk}_{\text{FAEST}}, sk, \sigma') = 0$  then return false
16:   return  $M'$ 
17: end function

```

5.2 Security Analysis

The construction inherits key-uniqueness, index-hiding, and forward-secrecy (Theorems 1 to 3) from Section 3. The additional FAEST signature provides:

Theorem 5 (Authenticity). *No PPT adversary can produce an auditor-accepted transcript $(C, \mathcal{I}, \sigma)$ without knowledge of the real session key sk , except with negligible probability.*

Proof (Sketch). Assume for the sake of contradiction that a PPT adversary $\mathcal{A}$ produces a valid auditor-accepted transcript $(C, \mathcal{I}, \sigma)$ without knowing the real session key sk . We construct a reduction that breaks the FAEST signature scheme. Let the challenger’s public key pk_{FAEST} be used as the public parameter in PQ-STAR. If $\mathcal{A}$ forges a transcript, then σ is a valid FAEST signature on the message sk that was never signed by the legitimate key holder. This yields an existential forgery against FAEST’s existential unforgeability under chosen-message attack (EUF-CMA) [5]. Therefore, $\mathcal{A}$ can succeed only with negligible probability. $\square$

Theorem 6 (Zero-Knowledge Privacy). *For every PPT auditor $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ whose output is computationally indistinguishable from $\mathcal{A}$ ’s real view of any transcript.*

Proof (Sketch). Given any auditor $\mathcal{A}$, we construct a simulator $\mathcal{S}$ that generates a fake transcript indistinguishable from a real execution. The simulator $\mathcal{S}$ first selects an arbitrary session key sk' (e.g., a random n -bit string). It then invokes the zero-knowledge simulator of the FAEST signature scheme to produce a simulated signature σ' on sk' (without using any secret key). Next, $\mathcal{S}$ encrypts some dummy message under sk' to obtain a ciphertext C . Finally, $\mathcal{S}$ outputs the transcript $(C, \mathcal{I}, \sigma')$, where $\mathcal{I}$ is any index tuple (which can be chosen arbitrarily, since it is public). By the zero-knowledge property of FAEST, the simulated σ' is indistinguishable from a real signature on sk' , and by the IND-CPA security [5], the ciphertext C reveals no information about the underlying message. Thus, $\mathcal{S}$ ’s output is computationally indistinguishable from $\mathcal{A}$ ’s real view of an honest transcript. $\square$

Consequently, the zero-knowledge variant achieves verifiable correctness of every session key while maintaining complete confidentiality of the master array.

At last, we elaborate on our choice to use FAEST rather than *zero-knowledge middleboxes* (ZKMBs). A natural alternative to using a digital signature in the zero-knowledge variant is to employ a general-purpose zero-knowledge proof of correct key derivation. Such proofs have been studied in the setting of ZKMBs [21], which allow a network middlebox to verify that a client’s encrypted TLS session complies with a policy without decrypting the traffic. In their construction, Grubbs *et al.* demonstrate that while a middlebox can verify a proof of correct TLS key derivation in roughly 2-5 ms, generating the proof on the client side takes several seconds and can require hundreds of megabytes of memory. Such heavy, interactive proofs are unsuitable for a rekeying primitive that runs

for every message. The FAEST signature scheme, on the other hand, implements a non-interactive zero-knowledge proof of knowledge of the secret session key: the signature on sk is constant-size (4.5-5.9 kB, or 3.9 kB for an optimized variant) and can be verified in a few million cycles [5]. By instantiating the zero-knowledge PQ-STAR variant with FAEST, we obtain post-quantum soundness together with compact, efficient proofs, avoiding the prohibitive client-side costs of general zero-knowledge proof systems.

6 Performance and Parameter Selection

Our design aims to balance security and efficiency by choosing appropriate parameters (n, m, k) and ensuring that the overhead of hashing and storage is practical.

Parameter selection. PQ-STAR is parameterised by three integers: n , m and k . Each array entry is an n -bit string; the array comprises m such entries; and each session uses a tuple of k indices. For example, for post-quantum security, one may choose $n = 256$, so that the session key has 256 bits and offers roughly 128 bits of quantum security [20]. A master array with m entries then contains $m \cdot n$ bits. For instance, taking $m = 2^{20}$ yields an array of $2^{20} \cdot 256$ bits, or 32 MiB; Section 3 explains how to avoid storing this in full by deriving entries from short seeds. Each session independently samples k indices from $[m]$; the probability that two sessions choose the same tuple is m^{-k} . Selecting $k = 16$ and $m = 2^{20}$, for example, yields a negligible chance of any collision even across billions of sessions: for T sessions, the probability of a collision is at most $\binom{T}{2}/m^k$, which remains negligible even for very large T .

Computational cost. Deriving a session key requires computing a single hash of k selected array entries. Since each entry contains n bits, the input to the hash has $k \cdot n$ bits, or $k \cdot n/8$ bytes. With $n = 256$ and $k = 16$, the hash input is only 512 bytes. Hash functions such as SHA-256 process data at gigabit rates [38]; hashing a few hundred bytes therefore takes on the order of microseconds, which is negligible compared with the time to encrypt a message. PQ-STAR performs exactly one hash per session, regardless of the message length. Consequently, the overhead of session-key derivation is constant per message (dominated by the underlying AEAD encryption and independent of the message length) [18].

Deployment Considerations. PQ-STAR can be integrated into existing protocols with minimal changes. The index tuple $\mathcal{I}$ can be sent in the clear or as associated data in the AEAD; because it carries no secret information, it does not require encryption. PQ-STAR operates at the application layer and can run atop a secure transport such as TLS 1.3. TLS 1.3 negotiates symmetric keys and provides confidentiality and integrity [37], but it does not support public auditability. Integrating PQ-STAR with TLS 1.3 thus allows Alice and Bob to derive auditable session keys while retaining the confidentiality and integrity guarantees of the transport.

Comparison with Existing Techniques. To our knowledge, there is no prior work that simultaneously combines stateless rekeying, post-quantum security, and public auditability (see, e.g., [1,2,10,24,25,26,33,39,41]). Existing rekeying schemes either rely on stateful rekeying or lack an audit mechanism. Because PQ-STAR introduces new functionality rather than optimizing an existing primitive, our evaluation focuses on qualitative properties (statelessness, post-quantum soundness, and auditability) rather than quantitative benchmarks.

7 Conclusion

We introduced PQ-STAR, a post-quantum, stateless, auditable rekeying scheme. PQ-STAR allows each message to be encrypted with a fresh session key derived from a shared secret array, while enabling an authorized auditor to verify any ciphertext by revealing only the corresponding one-time session key. The scheme utilizes only hash functions, operates without maintaining state between messages, and does not require any synchronization between sender and receiver. We proved that all three PQ-STAR variants are secure. The commitment-based PQ-STAR variant augments the plain construction with binding commitments, preventing a malicious sender from later claiming an unrelated session key. The Zero-knowledge PQ-STAR variant equips each session key with a signature-based zero-knowledge proof, which proves that the session key was derived honestly, without ever revealing the secret preimages.

References

1. Abdalla, M., Belaïd, S., Fouque, P.A.: Leakage-resilient symmetric encryption via re-keying. In: Cryptographic Hardware and Embedded Systems-CHES 2013: 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings 15. pp. 471–488. Springer (2013)
2. Abdalla, M., Bellare, M.: Increasing the lifetime of a key: a comparative analysis of the security of re-keying techniques. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 546–559. Springer (2000)
3. Ali, S., Guo, X., Karri, R., Mukhopadhyay, D.: Fault attacks on aes and their countermeasures. In: Secure System Design and Trustable Computing, pp. 163–208. Springer (2016)
4. Barker, E.B., Kelsey, J.M., et al.: Recommendation for random number generation using deterministic random bit generators (revised). US Department of Commerce, Technology Administration, National Institute of ... (2007)
5. Baum, C., Beullens, W., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., et al.: Faest v2: Algorithm specifications (2025)
6. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The sphincs+ signature framework. In: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security. pp. 2129–2146 (2019)

7. Biryukov, A., Khovratovich, D.: Related-key cryptanalysis of the full aes-192 and aes-256. In: International conference on the theory and application of cryptology and information security. pp. 1–18. Springer (2009)
8. Biryukov, A., Khovratovich, D., Nikolić, I.: Distinguisher and related-key attack on the full aes-256. In: Annual International Cryptology Conference. pp. 231–249. Springer (2009)
9. Chan, J., Rogaway, P.: On committing authenticated-encryption. In: European Symposium on Research in Computer Security. pp. 275–294. Springer (2022)
10. Chang: Hop (2025), manuscript in preparation
11. Chen, L., Chen, L.: Recommendation for key derivation using pseudorandom functions. US Department of Commerce, National Institute of Standards and Technology (2024)
12. Chen, L., Chen, L., Jordan, S., Liu, Y.K., Moody, D., Peralta, R., Perlner, R.A., Smith-Tone, D.: Report on post-quantum cryptography, vol. 12. US Department of Commerce, National Institute of Standards and Technology ... (2016)
13. Coron, J.S., Goubin, L.: On boolean and arithmetic masking against differential power analysis. In: Cryptographic Hardware and Embedded Systems—CHES 2000: Second International Workshop Worcester, MA, USA, August 17–18, 2000 Proceedings 2. pp. 231–237. Springer (2000)
14. Denis, F., Lucas, S.: The AEGIS Family of Authenticated Encryption Algorithms. Internet-Draft draft-irtf-cfrg-aegis-aead-17, Internet Engineering Task Force (Aug 2025), <https://datatracker.ietf.org/doc/draft-irtf-cfrg-aegis-aead/17/>, work in Progress
15. Dolev, S.: System and method for Merkle puzzles symmetric key establishment and generation of Lamport Merkle signatures (May 2019), uS Patent 0140819
16. Dolev, S., Panagopoulou, P.N., Rabie, M., Schiller, E.M., Spirakis, P.G.: Rationality authority for provable rational behavior. Algorithms, Probability, Networks, and Games: Scientific Papers and Essays Dedicated to Paul G. Spirakis on the Occasion of His 60th Birthday pp. 33–48 (2015)
17. Dolev, S., Yagudaev, A., Yung, M.: Hbss:(simple) hash-based stateless signatures—hash all the way to the rescue! Cryptography and Communications pp. 1–18 (2025)
18. Dworkin, M.J.: Sp 800-38d. recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. National Institute of Standards & Technology (2007)
19. Dziembowski, S., Pietrzak, K.: Leakage-resilient cryptography. In: 2008 49th Annual IEEE Symposium on Foundations of Computer Science. pp. 293–302. IEEE (2008)
20. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. pp. 212–219 (1996)
21. Grubbs, P., Arun, A., Zhang, Y., Bonneau, J., Walfish, M.: {Zero-Knowledge} middleboxes. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 4255–4272 (2022)
22. Grubbs, P., Lu, J., Ristenpart, T.: Message franking via committing authenticated encryption. In: Annual International Cryptology Conference. pp. 66–97. Springer (2017)
23. Kelsey, J., Chang, S., Perlner, R.: Sha-3 derived functions: cshake, kmac, tuplehash, and parallelhash. NIST special publication **800**, 185 (2016)
24. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Bar-

- bara, California, USA, August 15–19, 1999 Proceedings 19. pp. 388–397. Springer (1999)
25. Kocher, P., Jaffe, J., Jun, B., Rohatgi, P.: Introduction to differential power analysis. *Journal of Cryptographic Engineering* **1**, 5–27 (2011)
26. Kocher, P.C.: Leak-resistant cryptographic indexed key update (Mar 25 2003), uS Patent 6,539,092
27. Krawczyk, H.: Cryptographic extraction and key derivation: The hkdf scheme. In: Annual Cryptology Conference. pp. 631–648. Springer (2010)
28. Krawczyk, H., Eronen, P.: Hmac-based extract-and-expand key derivation function (hkdf). Tech. rep. (2010)
29. Lamport, L.: Constructing digital signatures from a one way function (1979)
30. Langford, S.K., Hellman, M.E.: Differential-linear cryptanalysis. In: Advances in Cryptology—CRYPTO’94: 14th Annual International Cryptology Conference Santa Barbara, California, USA August 21–25, 1994 Proceedings 14. pp. 17–25. Springer (1994)
31. Len, J., Grubbs, P., Ristenpart, T.: Partitioning oracle attacks. In: 30th USENIX security symposium (USENIX Security 21). pp. 195–212 (2021)
32. Mangard, S., Oswald, E., Popp, T.: Power analysis attacks: Revealing the secrets of smart cards, vol. 31. Springer Science & Business Media (2008)
33. Medwed, M., Standaert, F.X., Großschädl, J., Regazzoni, F.: Fresh re-keying: Security against side-channel and fault attacks for low-cost devices. In: Progress in Cryptology—AFRICACRYPT 2010: Third International Conference on Cryptology in Africa, Stellenbosch, South Africa, May 3–6, 2010. Proceedings 3. pp. 279–296. Springer (2010)
34. Mennink, B.: Beyond birthday bound secure fresh rekeying: Application to authenticated encryption. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 630–661. Springer (2020)
35. Nir, Y., Langley, A.: ChaCha20 and Poly1305 for IETF Protocols. RFC 8439 (Jun 2018). <https://doi.org/10.17487/RFC8439>, <https://www.rfc-editor.org/info/rfc8439>
36. Renaud, M., Standaert, F.X., Veyrat-Charvillon, N.: Algebraic side-channel attacks on the aes: Why time also matters in dpa. In: International Workshop on Cryptographic Hardware and Embedded Systems. pp. 97–111. Springer (2009)
37. Rescorla, E.: The transport layer security (tls) protocol version 1.3. Tech. rep. (2018)
38. Santos Jr, C.E., Silva, L.M.d., Torquato, M.F., Silva, S.N., Fernandes, M.A.: Sha-256 hardware proposal for iot devices in the blockchain context. *Sensors* **24**(12), 3908 (2024)
39. Smyshlyaev, S.V.: Re-keying Mechanisms for Symmetric Keys. RFC 8645 (Aug 2019). <https://doi.org/10.17487/RFC8645>, <https://www.rfc-editor.org/info/rfc8645>
40. Tiri, K., Verbauwhede, I.: A logic level design methodology for a secure dpa resistant asic or fpga implementation. In: Proceedings Design, Automation and Test in Europe Conference and Exhibition. vol. 1, pp. 246–251. IEEE (2004)
41. Ueno, R., Homma, N., Inoue, A., Minematsu, K.: Fallen sanctuary: a higher-order and leakage-resilient rekeying scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(1), 264–308 (2024)